

Mini-projets

1) Docker Compose

Dans les environnements modernes, la supervision des systèmes et des applications est essentielle pour garantir leur performance et leur disponibilité.

La stack **TIG** (Telegraf, InfluxDB, Grafana) est une solution largement utilisée pour la collecte, le stockage et la visualisation de métriques.

Docker et Docker Compose permettent de déployer rapidement des architectures multi-services de manière reproductible et portable. (voir les notes sur eCampus)

2) Kubernetes

La *Voting App* est une application distribuée composée de plusieurs microservices (frontend, backend, base de données, cache). Elle est couramment utilisée pour illustrer les concepts de conteneurisation et d'orchestration avec Kubernetes.

Vous allez déployer cette application sur un cluster Kubernetes et vérifier son bon fonctionnement.

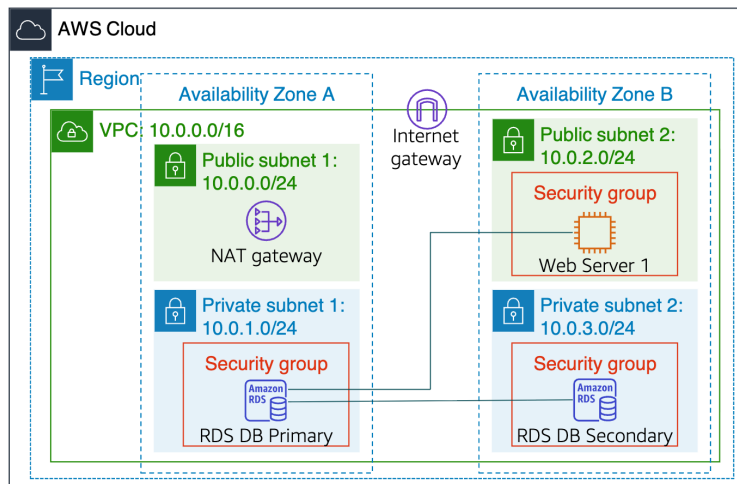
3) Terraform

Vous êtes chargé(e) de définir une infrastructure cloud sur AWS destinée à héberger une application web hautement disponible.

L'architecture cible est répartie sur **deux Availability Zones** et comprend :

- Un réseau virtuel (VPC)
- Des sous-réseaux publics et privés
- Un serveur web
- Une base de données RDS avec réplication
- Les composants réseau nécessaires (Internet Gateway, NAT Gateway, Security Groups)

L'ensemble de l'infrastructure devra être **décrit et déployé exclusivement à l'aide de Terraform**



4) Ansible

Vous allez utiliser **Ansible**, un outil d'automatisation et de gestion de configuration, afin de déployer et configurer automatiquement un serveur **Apache HTTP** sur une ou plusieurs machines distantes.

L'objectif est de remplacer les configurations manuelles par une approche **Infrastructure as Code (IaC)**, reproductible et maintenable.

5) GitHub actions

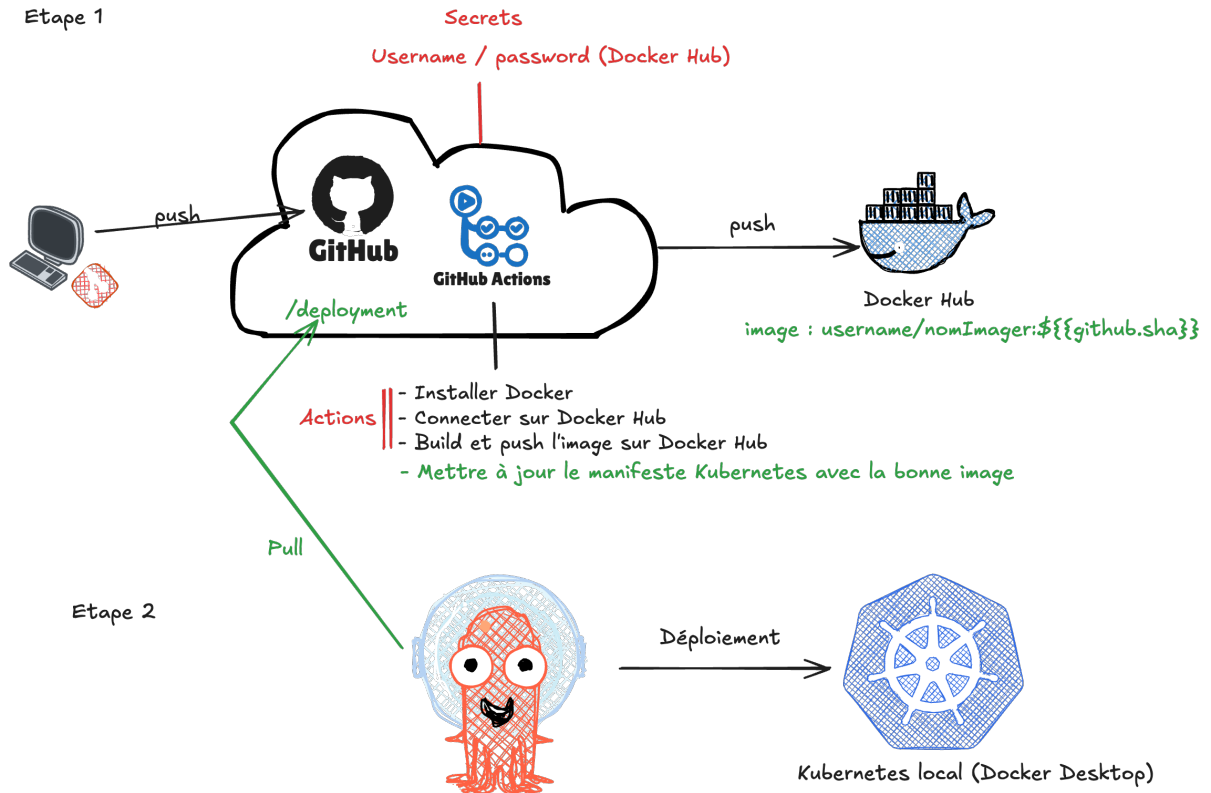
a) l'objectif de ce travail est de mettre en place un **workflow GitHub Actions** permettant d'automatiser l'ensemble du cycle de déploiement d'une application conteneurisée.

Vous devrez concevoir un pipeline CI/CD qui réalise les étapes suivantes :

- Construire automatiquement une **image Docker** de l'application
- Publier (*push*) l'image Docker sur **Docker Hub**
- Versionner l'image à l'aide d'un tag unique (par exemple le SHA du commit)
- Déployer automatiquement l'application sur un cluster **Kubernetes** en s'appuyant sur **Argo CD**

Le workflow devra être déclenché à chaque `push` sur le dépôt GitHub et utiliser des **secrets GitHub** pour gérer les informations sensibles (identifiants Docker Hub).

Etape 1

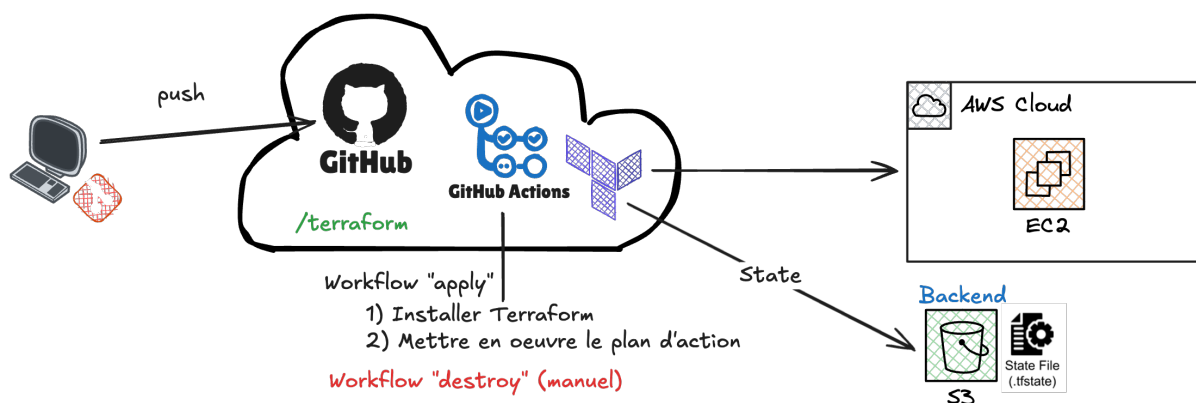


Etape 2

b) l'objectif de cette partie est de mettre en place une **infrastructure cloud automatisée** et de la gérer via des **workflows GitHub Actions** en s'appuyant sur **Terraform**.

Vous devrez concevoir et configurer **deux workflows distincts** :

- **Workflow de création (apply)**
Déployer automatiquement une infrastructure basée sur **Amazon EC2** à l'aide de **Terraform**.
- **Workflow de destruction (destroy)**
Détruire l'infrastructure EC2 précédemment créée



L'état de l'infrastructure (**terraform state**) devra être stocké de manière centralisée et persistante en utilisant un **bucket Amazon S3 configuré comme backend Terraform**, afin de garantir la cohérence de l'état et permettre une exécution fiable des workflows depuis GitHub Actions.